Experiment 2: Perceptron Learning

Objectives:

1- Understand the perceptron learning algorithm.
2- Implement the perceptron learning algorithm using Python.
3- Run real-world applications in Python.

Introduction:

The idea of a Perceptron is analogous to the operating principle of the basic processing unit of the brain — Neuron. A Neuron is comprised of many input signals carried by Dendrites, the cell body, and one output signal carried along Axon. The Neuron fires an action signal when the cell meets a particular threshold. This action either happens or they don't; there is no such thing as a "partial" firing of a neuron.

Similarly, the perceptron has many inputs(often called features) that are fed into a Linear unit that produces one binary output. Therefore, perceptrons can be applied in solving Binary Classification problems where the sample is to be identified as belonging to one of the predefined two classes.

The Algorithm

Since Perceptron's are Binary Classifiers (0/1), we can define their computation as follows:

$$f_\Phi(\mathbf{x}) = \begin{cases} 1 & \text{if } b + \mathbf{w} \cdot \mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Let's recall that the dot product of two vectors of length n ($1 \le i \le n$) is

$w \,.\, x = \sum_i w_i \,.\, x_i$

The function f(x)=b+w.x is a linear combination of weight and feature vectors. Perceptron is, therefore, a linear classifier — an algorithm that predicts using a linear predictor function.

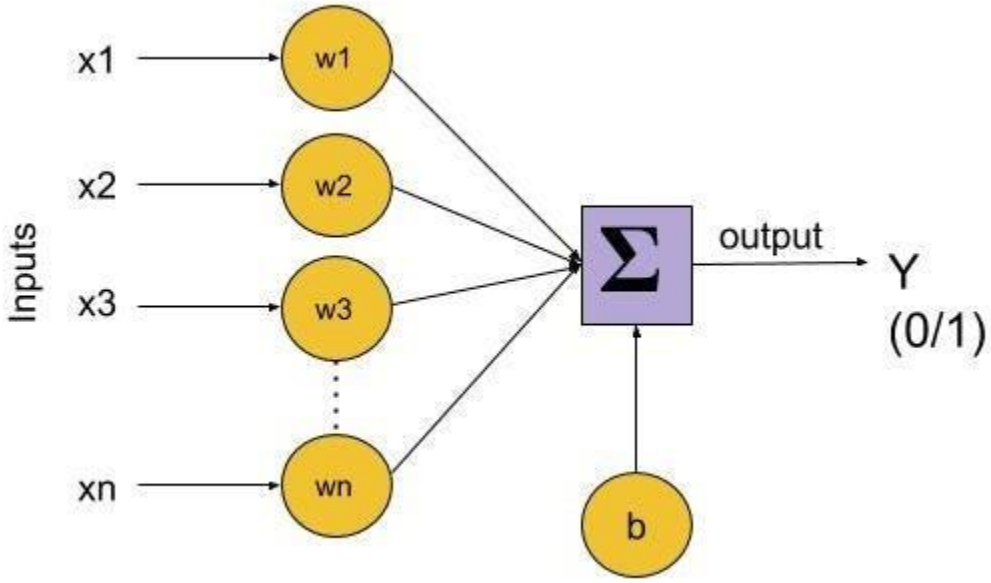


Figure 1

The weights signify the effectiveness of each feature x_i in x on the model's behavior. Higher the weight w_i of a feature x_i, higher is it's influence on the output. On the other hand, the bias 'b' is like the intercept in the linear equation. It's a constant that helps the model adjust in a way that best fits the data. The bias term assumes an imaginary input feature coefficient x_0=1.

Experiments

Experiment 1: Learning AND gate

The Following code implements the AND gate learning in Python

```
import numpy
import random

w1 = random.uniform(-0.5, 0.5)
w2 = random.uniform(-0.5, 0.5)
threshhold = random.uniform(-0.5, 0.5)

training_x = numpy.asarray([[0,0], [0,1], [1,0], [1,1]])
out = [0,0,0,1]

def positive(number):
    if(number >= 0):
        return 1
    else:
        return 0

error = numpy.array([0,0,0,0])
for j in range(len(training_x)):
    check = positive(numpy.dot(numpy.asarray([w1,w2]), training_x[j]) +
threshhold)
    error[j] = out[j] -check
errornumber = numpy.sum(error)


iterations = 1000
count = 1
eta = 0.1

values = [w1, w2, threshhold]
while count < iterations and errornumber != 0:
    for j in range(len(training_x)):
        check = positive(numpy.dot(numpy.asarray([w1,w2]),
training_x[j]) + threshhold)
        error[j] = out[j] -check
        w1 = values[0] + eta * error[j]*training_x[j][0]
        w2 = values[1] + eta * error[j]*training_x[j][0]
        threshhold = values[2] + eta*training_x[j][0]
    values = [w1, w2, threshhold]
    errornumber = numpy.sum(error)

    print("ERRORS: " + str(errornumber))
    count += 1

print("w1 " + str(values[0]) + "w2 " + str(values[1]) + "theta " +
str(values[2]))

print("count " + str(count))
```

why

Run the above code and discuss the effect of:

- Changing the learning rate to 0.01
- Changing the initial wight value range to -5 to 5
- Plot the straight line presenting the output.
- Change the logical function and run the program, then discuss the results and plot the resulting line after training.

Experiments 2

Another code for programming the perceptron using AND gate is as follows

```
import random
trainingset = [[0,0,0], [0,1,0], [1,0,0], [1,1,1]]
eta = 0.3
maxiterations = 100
w1 = random.uniform(-0.2, 0.2)
w2 = random.uniform(-0.2, 0.2)
w0 = random.uniform(-0.2, 0.2)
error = random.uniform(-0.2, 0.2)
count = 0
while count < maxiterations and error != 0:
    error = 0
    for array in trainingset:
        target = array[2]
        output = 0
        summation = w1*array[0] + w2*array[1] - w0
        if(summation > 0):
            output = 1
        else:
            output = 0

        if(output != target):
            error += 1

        w1 += eta*(target - output)*array[0]
        w2 += eta*(target - output)*array[1]
        w0 += eta*(target - output)*(-1)




        print("output " + str(output) + " target " + str(target))
        print("ERROR " + str(error))
    count += 1
print("COUNT " + str(count))
print("ENDING ERROR" + str(error))
print("w1 " + str(w1) + "w2 " + str(w2) + "w0 " + str(w0))
```

Experiment 3:

In this experiment the IRIS dataset is classified by the perceptron, to visualize the dataset, lets first run the following code:

- **Numpy:** this is one of the most common libraries used in data applications, it allows us to create 'numpy arrays' which are similar to lists but their implementation allows us to performs computations on these arrays.
- **Random:** we'll be using random to initialize our weights.
- **Pandas:** pandas is also a very powerful library for dealing with data, but in our case we'll be just using pandas to import our sample dataset.
- **Matplotlib:** lastly we'll be using matplotlib to help us visualize concepts.

For this example, we'll be using the 'iris flower' dataset. This data has 3 different flowers with 50 different samples for each flower type containing 4 features that describe the flower numerically (i.e petal length).

To keep things simple we'll be using 2 out of the 3 flowers, so our perceptron will essentially predict wether a flower is a Setosa or a Versicolor based on two features (i.e petal width and length). From the graph it's easily noticable that **our data can seperated by a linear line**, and this is what our model will attempt to learn. Note that since we're keeping it basic the model we're going for will only be able to classify linearly seperable data.here we define a new class "perceptron", initializing the learning rate **(eta)** & number of iterations **(n_iter)**. These two values are called hyperparameters, the learning rate is a float from 0 to 1, while the number of iterations is an integer. The optimal value for these two variables depends on the

problem itself. Hyperparameter optimisation is a topic for another article, for now all we need to know about these two is the following:

- **Learning rate (eta):** The intuition here is that if the learning rate is too high our model will overshoot.
- **Number of iterations (n_iter):** This is the amount of iterations that our algorithm will run over the training data.

There's three functions defined in our perceptron model. The **fit()** function is basically where all the training happens and the weights get updated iteratively so that after we finish training we should have an array of size 3 containing the weights for the features and the last value is the 'bias' which is the equivelent to the 'y-intercept'.

The fit function consists of two nested loops. The outer loop is how many times the algorithm will run over our training set, and the inner loop is the actual 'running over training set'. Here, the 'X' parameter is a 2-d numpy array containing the feature values and 'y' is the label for each row of features. We use the built-in python keyword **zip(X, y)** to basically go over each labelled sample.

The most integral component to our learning process is the weight update. The update is defined as:

$$Update = eta * (label - prediction)$$

This can be thought of as the mathematical representation of how correct or incorrect our model's prediction was. To understand this further, we can look at the **four possible outcomes** from this equation. As we previously mentioned, each flower is expressed in a numerical value of either 1 or -1, where -1 is 'Setosa' and 1 is 'Versicolor'.

now, in our inner loop the compiler will go over a row of samples and attempt to ‘predict’ what this sample is using it’s given features and our model’s current weights.

Taking the two scenarios where our model is **incorrect** (we’re ignoring the eta since it’s just a constant):

- (1)-(-1) = 2
- (-1)-(1) = -2

As for when our model is **correct** we get, in other word **both the label and prediction have the same value**:

- (1)-(1) = 0
- (-1)-(-1) = 0

When the prediction is wrong the weights get updated in the direction of the error. Whereas if it’s correct the result is zero and the weights are left unchanged.

Training our perceptron

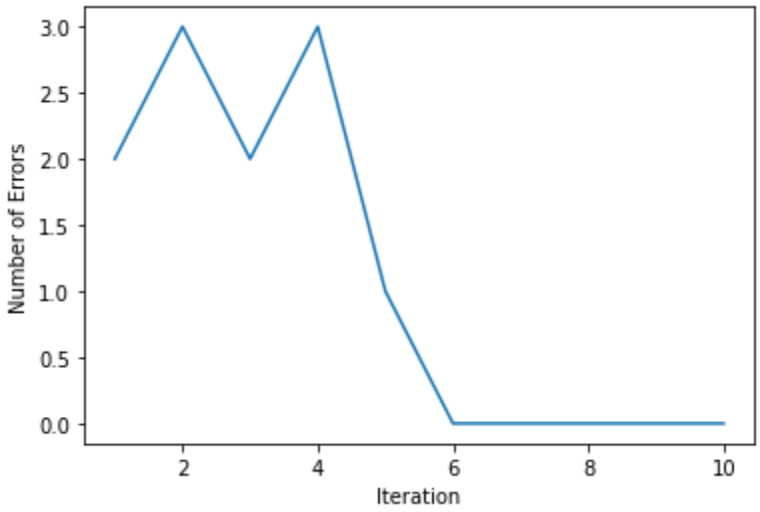
hopefully, by now you have an idea of what the model does, you also refer to the comments in the code if anything is unclear.

first, we initialize our perceptron as a new object. We already imported our dataset but here we define it in a format that can be understood by our **fit()** function (taking features in columns at index 0 and 2 as a 2-d numpy array & the 4th index which is the class of the sample in a seperate numpy array). We also use the function np.where to change the string values of the class to 1 (versicolor) and -1 (setosa).

Then we can run the **fit()** function on our X and y values to train our perceptron, finally after a certain number of iteration we should have a numpy array containing our optimal weight and can now call the **predict()** function on an unlabelled sample and have our perceptron classify it.

Plotting errors

The defined self.errors_ variable doesn't actually contribute to our learning process but we can use it to plot the number of errors against each iteration



Errors in each iteration

From the graph we can see that our model converged after the 6th iteration. This can slightly change with each run since we are randomizing our initial weights.

```
import numpy as np
import matplotlib.pyplot as plt
import random
import pandas as pd
#import dataset
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-
databases/iris/iris.data', header=None)

#get the first 50 rows (which are all setosas) with only the first 2
features (petal length & width)
setosa = np.array(df.iloc[0:50, [0,2]])
```

```
#get rows 50-100 (which are all versicolor) with only the first 2
features (petal length & width)
versicolor = np.array(df.iloc[50:100, [0,2]])

#plot points where blue o's denote setosa & red x's denote versicolor
plt.plot(setosa[:, 0], setosa[:, 1], "bo", label="Setosa")
plt.plot(versicolor[:, 0], versicolor[:, 1], "rx", label="Versicolor")
plt.xlabel("petal width")
plt.ylabel("petal length")
plt.legend(loc='upper left')
plt.show()
class Perceptron():
    #initialize hyperparameters (learning rate and number of
iterations)
    def __init__(self, eta=0.1, n_iter=50):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
            self.w_ = [random.uniform(-1.0, 1.0) for _ in range(1 +
X.shape[1])]  # randomly initialize weights
            self.errors_ = []  # keeps tracks of the number of errors
per iteration for observation purposes

            # iterate over labelled dataset updating weights for each
features accordingly
            for _ in range(self.n_iter):
                errors = 0
                for xi, label in zip(X, y):
                    update = self.eta * (label - self.predict(xi))
                    self.w_[1:] += update * xi
                    self.w_[0] += update
                    errors += int(update != 0.0)
                self.errors_.append(errors)
            return self

        # compute the net input i.e scalar sum of X and the weights
plus the bias value
    def net_input(self, X):
            return np.dot(X, self.w_[1:]) + self.w_[0]

        # predict a classification for a sample of features X
    def predict(self, X):
            return np.where(self.net_input(X) >= 0.0, 1, -1)
#preparing our data to be understood by our model
X = df.iloc[0:100, [0,2]].values
y = df.iloc[0:100, 4].values
y = np.where(y == 'Iris-setosa', -1, 1)

ppn = Perceptron(n_iter=10) #initializing a new perceptron
ppn.fit(X, y) #training
#plotting errors against number of iterations
plt.plot(range(1, len(ppn.errors_) + 1), ppn.errors_)
plt.xlabel("Iteration")
plt.ylabel("Number of Errors")
plt.show()
```

1) modify the above code, to classify the third class (Virginica) of the iris flower with:
   a) The first class setosas
   b) The second class versicolor

   Discuss your results and visualize them.

2) Train the perceptron to the classification problem for class 1 and class 2 but using the third and fourth features.
   Discuss your result.

3) Change the learning rate of the perceptron and discuss your results